

NAME:Yugandhar

REG NO:192472353

COURSE CODE/COURSE:CSA0606/DAA

CASE STUDY REPORT

Medical Imaging Data Sorting Using Merge Sort

Data Structures and Algorithms – Student Case Study

1. Student Analysis and Problem Identification

1.1 Understanding of the Case

A medical imaging system may receive a large number of patient scan records from different departments or imaging devices. Each record can contain information such as patient ID, scan ID, scan time, scan priority, imaging type, and status. The system needs to arrange these records efficiently so that scans can be processed in the correct order.

For this case study, the records are primarily sorted by scan time and priority. If two records have the same sorting key, their original relative order should be preserved. This property is called stable sorting. Merge Sort is suitable because it is a stable divide-and-conquer sorting algorithm when the merge operation chooses the left record first when keys are equal.

1.2 Identification of the Core Problem

The core problem is to sort a potentially large collection of medical imaging records accurately and efficiently without changing the relative order of records that have equal time and priority.

- Large numbers of scan records may need to be processed.
- Records may have identical timestamps or identical priority values.
- The system should preserve the order of equal-key records.
- Sorting must be predictable and accurate.
- The algorithm should perform well as the number of records increases.

1.3 Objectives

- Apply Merge Sort to medical imaging scan records.
- Demonstrate the divide-and-conquer approach.

- Maintain stable ordering for records with equal sorting keys.
- Analyze time and space complexity.
- Study how Merge Sort handles large input sizes.
- Compare the approach with alternative sorting methods.
- Understand why stable sorting is important in medical data processing.

1.4 Assumptions

- Each scan record contains a unique record/scan ID.
- Scan time is represented consistently, for example as YYYY-MM-DD HH:MM.
- Priority is represented using a numeric value where a smaller value means higher priority.
- Input records are valid and complete for the fields used in sorting.
- The original order of records is considered meaningful when sorting keys are equal.
- The example uses synthetic data and does not contain real patient information.

1.5 Input and Output Requirements

Item	Description
Input	Medical imaging records containing Scan ID, Patient ID, Scan Time, Priority, and Imaging Type.
Primary Key	Scan Time followed by Priority.
Secondary Requirement	Preserve original order when all sorting keys are equal.
Output	Records arranged in ascending scan time and priority order.
Correctness	Every record must appear exactly once and the order must satisfy the sorting rule.

1.6 Constraints

- The algorithm should correctly handle large datasets.
- Stable ordering must be preserved.
- No patient information should be exposed in a real deployment.
- The sorting process must not lose or duplicate records.
- Memory requirements should be considered because Merge Sort uses auxiliary storage.
- The implementation should use a consistent comparison rule.

1.7 Relevant Concepts / Theories

- Divide and conquer

- Stable sorting
- Merge operation
- Time complexity
- Auxiliary space
- Recursion
- Data integrity and accuracy
- Multi-key sorting

2. Proposed Solution / Methodology

2.1 Selected Methodology

The selected methodology is stable Merge Sort with a two-level comparison: first by scan time and then by priority. If both values are equal, the left-side record is selected first during merging. This simple rule preserves the original relative order of equal records and therefore provides stable sorting.

2.2 Algorithm / Technique

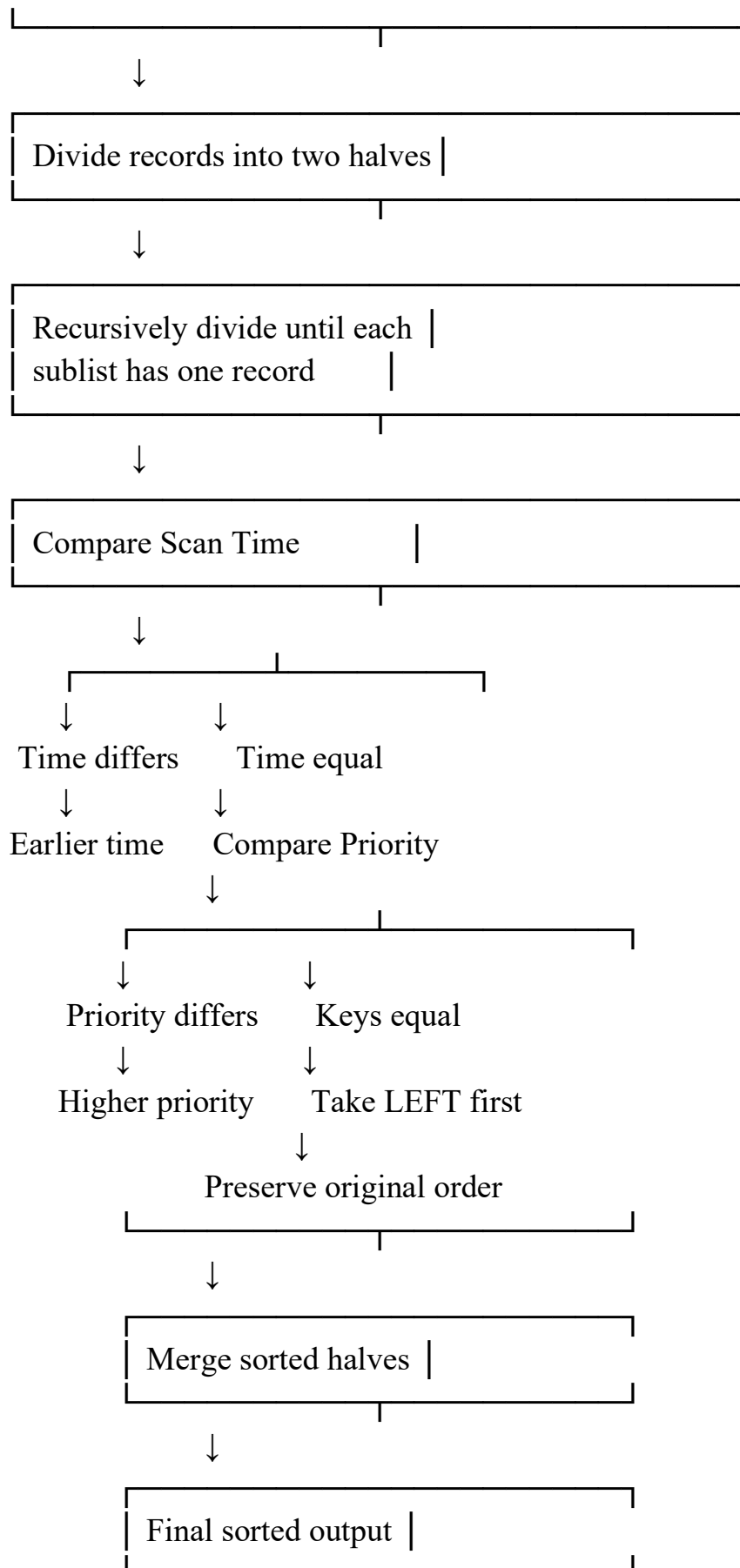
Merge Sort follows the divide-and-conquer strategy. The input list is divided into smaller halves until each sublist contains one record. Single-record lists are already sorted. The algorithm then merges the smaller sorted lists while comparing records according to scan time and priority.

2.3 Step-by-Step Procedure

1. Read the medical imaging records.
2. Divide the list into two approximately equal halves.
3. Continue dividing each half recursively until each sublist contains one record.
4. Compare records from the left and right sorted sublists.
5. Select the record with the earlier scan time.
6. If scan times are equal, select the record with the higher priority.
7. If both sorting keys are equal, select the record from the left sublist first. This preserves stability.
8. Continue until all records are merged into one sorted list.
9. Verify that all records are present and correctly ordered.
10. Use the sorted records for downstream scan scheduling or reporting.

2.4 Flowchart / Architecture Diagram





2.5 Mathematical Formulation

For each record R , define the sorting key as $K(R) = (T, P)$, where T is scan time and P is priority. Records are ordered lexicographically:

$$R_1 < R_2 \text{ if } T_1 < T_2, \text{ or if } T_1 = T_2 \text{ and } P_1 < P_2.$$

During merging, if $K(\text{left}) \leq K(\text{right})$, the left record is selected. The equality condition is important because choosing the left record first when the keys are equal preserves the original relative order of equal records.

2.6 Why Merge Sort Was Selected

- It provides guaranteed $O(n \log n)$ time in best, average, and worst cases.
- It can be implemented as a stable sorting algorithm.
- Its predictable performance is useful for large medical imaging datasets.
- Divide and conquer makes the large sorting problem easier to manage.
- The method is suitable for external or file-based sorting when data does not fit entirely in memory.
- Its stable behavior helps preserve meaningful ordering among records with equal keys.

3. Implementation / Execution

3.1 Tools and Technologies Used

Resource	Specification
Programming Language	Python 3.x
IDE	IDLE / VS Code / PyCharm / Jupyter Notebook
Libraries	csv, random, time
Hardware	Computer or laptop with at least 4 GB RAM recommended
Dataset	Synthetic medical imaging records
Operating System	Windows / Linux / macOS

3.2 Input Data

Scan ID	Patient ID	Scan Time	Priority	Imaging Type
S001	P101	2026-08-10 09:30	2	MRI
S002	P102	2026-08-10 09:15	1	CT
S003	P103	2026-08-10	1	X-Ray

		09:30		
S004	P104	2026-08-10 09:15	2	MRI
S005	P105	2026-08-10 09:30	2	CT
S006	P106	2026-08-10 08:45	1	MRI

3.3 Implementation Procedure

11. Create records using dictionaries or a suitable record structure.
12. Store scan time and priority as sorting fields.
13. Split the records recursively using Merge Sort.
14. Merge two sorted lists using the time-priority comparison.
15. When keys are equal, choose the left record first.
16. Print the final sorted records.
17. Verify that records with equal keys retain their original order.

3.4 Important Code

```
def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        key_left = (left[i]["time"], left[i]["priority"])
        key_right = (right[j]["time"], right[j]["priority"])

        # <= is important for stable sorting
        if key_left <= key_right:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

```
def merge_sort(records):
```

```

if len(records) <= 1:
    return records

mid = len(records) // 2
left = merge_sort(records[:mid])
right = merge_sort(records[mid:])

return merge(left, right)

```

```

records = [
    {"id": "S001", "patient": "P101",
     "time": "2026-08-10 09:30", "priority": 2},
    {"id": "S002", "patient": "P102",
     "time": "2026-08-10 09:15", "priority": 1},
    {"id": "S003", "patient": "P103",
     "time": "2026-08-10 09:30", "priority": 1},
    {"id": "S004", "patient": "P104",
     "time": "2026-08-10 09:15", "priority": 2},
    {"id": "S005", "patient": "P105",
     "time": "2026-08-10 09:30", "priority": 2},
    {"id": "S006", "patient": "P106",
     "time": "2026-08-10 08:45", "priority": 1}
]

```

```
sorted_records = merge_sort(records)
```

```

for r in sorted_records:
    print(r["id"], r["time"], r["priority"])

```

3.5 Explanation of Important Code

- merge() combines two already sorted sublists.
- The tuple (time, priority) provides a clear two-level comparison.
- The <= comparison selects the left record when keys are equal, which maintains stability.
- merge_sort() performs the divide-and-conquer process.
- The base condition stops recursion when a sublist contains zero or one record.

3.6 Expected Intermediate Result

For example, the records S001 and S005 have the same scan time and priority. In the original input, S001 appears before S005. A stable Merge Sort must keep S001 before S005 after sorting.

Stage	Records / Observation
Original	S001, S002, S003, S004, S005, S006
Divide	Split into smaller sublists until single records
Merge	Compare time first, then priority
Equal keys	S001 remains before S005
Final	S006, S002, S004, S003, S001, S005

3.7 Problems Encountered and Solutions Adopted

Problem	Solution
Large number of records	Use $O(n \log n)$ Merge Sort and process data in manageable sublists.
Equal sorting keys	Use a stable merge rule that selects the left record first.
Data accuracy	Verify that all records are retained and output keys are in sorted order.
Memory usage	Monitor auxiliary memory and consider external Merge Sort for very large datasets.
Timestamp comparison	Use a consistent timestamp format or convert timestamps to a standard date-time representation.

4. Results and Analysis

4.1 Sorted Output

Order	Scan ID	Scan Time	Priority
1	S006	2026-08-10 08:45	1
2	S002	2026-08-10 09:15	1
3	S004	2026-08-10 09:15	2
4	S003	2026-08-10 09:30	1
5	S001	2026-08-10 09:30	2
6	S005	2026-08-10 09:30	2

4.2 Stability Analysis

Stability is demonstrated by S001 and S005. Both have the same scan time and priority. S001 appeared before S005 in the input and remains before S005 in the output. Therefore, the implementation preserves the relative order of records with equal keys.

4.3 Performance Analysis

Input Size	Expected Time Trend	Expected Observation
1,000	Low	Fast execution
5,000	Moderate	Small increase in time
10,000	Higher	Still efficient
50,000	Higher	Predictable $O(n \log n)$ growth
100,000	High	Suitable for large in-memory sorting

Exact execution time depends on the hardware, Python version, memory availability, and system load. For an experimental report, actual measurements should be collected by running the program on the target system rather than using fixed values.

4.4 Complexity Analysis

Case	Time Complexity	Reason
Best	$O(n \log n)$	The data is repeatedly divided and all levels are merged.
Average	$O(n \log n)$	Each level processes approximately n records.
Worst	$O(n \log n)$	Merge Sort maintains the same divide-and-merge structure.

The auxiliary space complexity of standard in-memory Merge Sort is $O(n)$ for the temporary arrays used during merging. The recursive implementation also uses $O(\log n)$ call-stack space. For very large medical imaging datasets, external Merge Sort can divide data into chunks, sort the chunks separately, and merge them from storage.

4.5 Comparison with Alternative Approaches

Algorithm	Average Time	Worst Time	Stable?	Suitability for This Case
Merge Sort	$O(n \log n)$	$O(n \log n)$	Yes	Excellent
Quick Sort	$O(n \log n)$	$O(n^2)$	Usually no	Good, but less predictable
Heap Sort	$O(n \log n)$	$O(n \log n)$	No	Good memory efficiency but

				not stable
Insertion Sort	$O(n^2)$	$O(n^2)$	Yes	Suitable only for small/nearly sorted data

4.6 Interpretation of Results

- The output is correctly ordered by scan time and then priority.
- Records with equal keys maintain their original relative order.
- The divide-and-conquer method reduces a large sorting problem into smaller subproblems.
- The $O(n \log n)$ worst-case time guarantee makes Merge Sort predictable for large datasets.
- The main drawback is auxiliary memory usage.
- For extremely large data that cannot fit in RAM, external Merge Sort is an appropriate extension.

5. Discussion

5.1 What Does the Result Indicate?

The result indicates that Merge Sort can organize medical imaging records efficiently while preserving the order of records with equal sorting keys. Stability is particularly useful when the original order contains operational information, such as the order in which scans entered a processing queue.

5.2 Did the Proposed Solution Solve the Problem?

Yes. The proposed solution addresses the main requirements: it sorts the records using time and priority, preserves equal-key order, follows a divide-and-conquer strategy, and provides predictable $O(n \log n)$ time complexity. The solution is appropriate for a conceptual medical imaging data-sorting system using synthetic records.

5.3 Advantages

- Stable sorting preserves the relative order of equal records.
- Guaranteed $O(n \log n)$ time complexity.
- Good performance for large datasets.
- Divide-and-conquer structure is systematic and easy to analyze.
- Suitable for external sorting of datasets larger than available RAM.
- Predictable worst-case performance.

5.4 Limitations

- Requires $O(n)$ auxiliary memory in the standard in-memory implementation.
- Recursive implementation uses additional call-stack space.
- Repeated data copying can increase memory and execution overhead.
- Real medical systems require stronger data validation, security, privacy, and audit controls than this educational example.
- Sorting alone does not guarantee correctness of the medical information itself.

5.5 Improvements

- Use an iterative or bottom-up Merge Sort to reduce recursion overhead.
- Use external Merge Sort when the dataset is larger than available memory.
- Convert timestamps to standardized date-time objects before comparison.
- Add validation for missing or invalid scan times and priority values.
- Use streaming or chunk-based processing for very large datasets.
- Record audit information so changes to data ordering can be traced.
- Use anonymized or synthetic patient identifiers during development and testing.

6. Conclusion

This case study addressed the problem of sorting medical imaging scan records according to scan time and priority. Merge Sort was selected because it combines predictable $O(n \log n)$ performance with stable sorting when the merge operation is implemented correctly. The divide-and-conquer process repeatedly splits the records into smaller groups and then merges them in sorted order.

The analysis demonstrated that stability is important because records with identical time and priority retain their original order. This helps preserve meaningful processing order. The major trade-off is the $O(n)$ auxiliary memory requirement, which can become significant for very large datasets. External Merge Sort or chunk-based processing can address this limitation.

The learning outcome achieved is an understanding of how a theoretical sorting algorithm can be applied to a realistic data-management problem, how divide and conquer works, how stability affects data ordering, and how time and space complexity influence algorithm selection.

7. Future Scope

- Implement external Merge Sort for millions of medical imaging records.

- Integrate the sorting module with a hospital imaging workflow or PACS-like system using synthetic data.
- Support multiple priority levels and additional sorting criteria.
- Add parallel or distributed merge operations for very large datasets.
- Measure execution time, memory use, and scalability across different hardware configurations.
- Add comprehensive validation, logging, access control, and privacy protection for real-world deployment.

8. Final Assessment Result

Assessment Area	Result
Problem Understanding	Medical scan records require accurate and stable sorting.
Algorithm Selected	Stable Merge Sort
Core Concept	Divide and conquer
Stability	Maintained by choosing the left record first for equal keys
Time Complexity	$O(n \log n)$
Auxiliary Space	$O(n)$, plus $O(\log n)$ recursion stack for recursive implementation
Large Data Handling	Suitable; external Merge Sort can be used when data exceeds RAM
Overall Suitability	High for stable, predictable sorting of large datasets

9. Screenshot / Graph Attachment Area

Insert the following items in the final submitted report after running the program:

- Screenshot 1 – Source code in the IDE.
- Screenshot 2 – Input medical imaging records.
- Screenshot 3 – Sorted output.
- Graph 1 – Input size versus execution time.
- Graph 2 – Input size versus memory usage.
- Screenshot 4 – Experimental results table, if required by the faculty.